2 Pointers STRINGS

Leet-344 Reverse String

T-O(n) S-O(1)

// reverse(s.begin(), s.end());

/// Start, end pointers
Start → 0
end → s.size()-1

Swap(start, end)
start++
end--

until start<end → while loop

Leet-125 Valid Palindrom

T-O(n) S-O(1)

```
int start = 0
int end = s.size()-1
while(start <= end)
    if(!isalnum(s[start])) {start++; continue;}
    if(!isalnum(s[end])) {end--; continue;}
    if(tolower(s[start]) != tolower(s[end])) return false;
    start++
    end--
return true;
```

Leet-680 Valid Palindrom II

T-O(n) S-O(1)

mujub How to pass string in a function

Create 1 more fn to check palindrom →

```
bool checkP(const string& s, int i, int j)
    while(i<=j)
        if(s[i] != s[j])
            return false
        i++
        j--
    return true
```

Main fn →

```
int start=0
int end = s.length()-1
while(start <= end)
    if(s[start] != s[end])
        return checkP(s, start+1, end) || checkP(s, start, end-1);
    start++
    end--
return true
```

We can put spaces & tolower condition like in above question.

Ajanta PRODUCTS Officer

L-844 Backspace String Compare

Better → T- O(m+n) S- O(m+n)

// update s into st by removing characters to be deleted using #

```
String st = "";
for(char c : s)
    if(c == '#')
        if(!st.empty())
            st.pop_back()
    else st.push_back(c);
```

} if # Coming then don't add it, remove the last element of st using pop only if # is not the 1st element means if st is empty don't pop anything simply skip to next char

// Similarly update t into tt

// Now Compare st & tt

```
if(st != tt){
    return false
}
```

Return true;

Optimal → T- O(m+n) S- O(1)

* Track from back side (←)

```
int i = s.size()-1;
int j = t.size()-1;
int skipS=0;
int skipT=0;
```

> 2 ptrs

*while(i>=0 || j>=0) → use OR not And as And will prevent reaching to last left element of one string if another get over & will return true on basis of pre lastly compared elements of both elements.

```
while(i >= 0)
    if(s[i] == '#')
        skipS++
        i--
    else if(skipS > 0)
        skipS--;
        i--;
    else break
```

→ Count as many time # Come

. Skip characters in string 's' due to #

→ skip as many char as # Come

→ break if letter Comes

```
while(j >= 0)
```

→ Similar steps for string 't'

// Compare valid characters

```
if(i>=0 && j>=0)
    if(s[i] != t[j]) return false
```

// Edge Case where elements of one string get fully over but another's doesn't.

```
*else if( i>=0 || j>=0) return false
```

```
i--;
j--;
```

Return True

Leet-5 Longest Palindrom Substring

Approaches → Brute force, Expand Around Center $O(n^2)$ $O(1)$, DP $O(n^2)$ $O(n^2)$ (Recursion + memoization)

Manacher's Algo $O(n)$, $O(n)$, Recursive → TLE (Time limit exceeded) ↑ Pure (simple recursion — $O(2^n)$)

↓

Check into Leetcode Solutions (discussion)

2 pointers : Expand Around Center → T- $O(n^2)$ S- $O(1)$

```
int n = S.size();
int sp = 0;          'starting point'
int maxl = 1;

for(int cent=0; cent<n ; cent++)
    int l = center;
    int r = center;
    while(l>=0 && r<n && S[l]=S[r])
        if (r-l+1 > maxl)
            sp=l;
            maxL = r-l+1
        }
        l--;
        r++;

    l = center
    r = center + 1
    same while condition as of odd

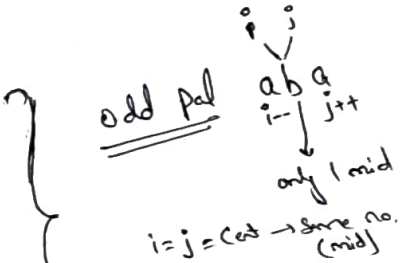
return S.substr(sp, maxlen)
```

Here we don't know from before itself that whether it's even pal or odd pal

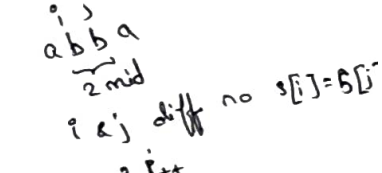
So, firstly it will work for odd
l = cent
r = cent
l-- r++
then it will go to even
l = cent
r = cent + 1
for same cent=0 then it will repeat both processes again & again for all cent from 0 to n

update SP only if new pal of greater length found.

odd pal



even pal



Leet- 647 Palindromic Substrings

→ Some approaches as above

	Time	Space
* Brute force →	$O(n^3)$	$O(1)$
* Recursion →	Exponential	$O(n)$ → TLE
* Memetization DP →	$O(n^2)$	$O(n^2)$
* Tabulation DP →	$O(n^2)$	$O(n^2)$
* Expand around Center →	$O(n^2)$	$O(1)$ → Better & Easy
* Manacher →	$O(n)$	$O(n)$ ✓✓ Best optimal